

Internal Architecture Specification

Module Structure, Component Interfaces, Internal APIs & Data Flow

Series	IOS+ Engineering Body, Document 1 of 6
Classification	CONFIDENTIAL — Internal Engineering Reference Enhanced NDA Required
Version	1.0
Date	May 2026
Prepared by	SMEPro Technologies Engineering
Contact	cmiguez@smeprotech.com
Audience	Principal Engineers, Platform Architects, Technical Co-Founders, NDA-Cleared Technical Evaluators
Status	Released — Authoritative Reference

Document Purpose

This is the internal implementation specification for the IOS+ system architecture. Where Volume I (Architecture Overview) describes *what* the system is and *why* it is designed as it is, EB Doc 1 specifies *how* it is built: module boundaries, internal

interfaces, data contracts between components, configuration schemas, and integration patterns. It is the authoritative reference for engineers building, extending, or integrating with IOS+.

Table of Contents

Section 0 — Engineering Body Series Index	2
Section 1 — Repository & Module Structure	3
1.1 Monorepo Layout	3
1.2 Package Dependency Graph	3
Section 2 — Middleware Engine — Module Specification	4
2.1 Entry Point & Transport Layer	4
2.2 Layer Orchestrator	5
2.3 Layer Module Interfaces (L1-L7)	5
2.4 Error Handling & Retry Policy	6
Section 3 — Gate 530 — Internal Module Specification	6
3.1 IPC Interface	6
3.2 Dimension Evaluation Engine	7
3.3 Session Context Cache	7
3.4 Quarantine Queue Integration	7
Section 4 — Evidence Fabric — Internal Module Specification	8
Section 5 — RAG Vault — Internal Module Specification	9
Section 6 — COS+ Database — Driver Specification	10
Section 7 — Shared Package — Type Definitions	11

Section 0 — Engineering Body Series Index	2
Section 8 — Internal API Surface	12
Section 9 — Configuration Schema	13
Section 10 — Document Lineage	14
Appendix A — Full InferenceResponse Schema	15
Appendix B — Protobuf Service Definitions	15
Appendix C — OpenAPI 3.1 Summary	16
Appendix D — Glossary	17

Section 0 — Engineering Body Series Index

The IOS+ Engineering Body comprises six classified technical documents. Each document addresses a distinct layer of the IOS+ system. Together they constitute the complete internal specification. All Engineering Body documents are classified **CONFIDENTIAL** and are available only to qualified technical evaluators under enhanced NDA. To request access to any document in this series, contact **cmiguez@smeprotech.com**.

Doc	Title	Scope & Coverage	Status
Doc 1 (<i>this document</i>)	Internal Architecture Specification	Module structure, component interfaces, internal APIs, data flow, configuration schemas	Released
Doc 2	Cryptographic Audit & Evidence Fabric	Ed25519 signing, JCS canonicalization,	Released

Doc	Title	Scope & Coverage	Status
	Specification	key lifecycle, Evidence Package format, WORM commitment protocol	
Doc 3	COS+ Database Schema & Role Architecture	Full DDL, role definitions, trigger code, partitioning strategy, migration runbook, audit log schema	Released
Doc 4	Gate 530 Compliance Dimension Matrix	All 127 dimensions: detection functions, scoring algorithms, thresholds, inter-dimension dependencies	Released
Doc 5	RAG Vault Retrieval Pipeline Specification	HNSW configuration, BM25 lexical layer, RRF fusion, context packing algorithm, ingestion pipeline	Released
Doc 6	Deployment, Operations & Recovery Runbook	Kubernetes manifests, Helm chart configuration, monitoring, alerting, incident response, DR procedures	Released

Access Control

All Engineering Body documents are classified CONFIDENTIAL. Distribution is restricted to NDA-cleared technical evaluators, principal engineers, platform architects, and technical co-founders. Unauthorized reproduction, transmission, or storage of this document is prohibited. Contact cmiguez@smeprotech.com to

initiate enhanced NDA onboarding.

Section 1 — Repository & Module Structure

1.1 Monorepo Layout

The IOS+ platform is maintained as a single monorepo. All services, infrastructure definitions, and documentation are co-located to enforce consistent versioning, shared type contracts, and atomic multi-package releases. The top-level structure is as follows:

```
ios-plus/ |— packages/ |   |— middleware-engine/           # IOS+ Middleware
Engine — L1-L7 orchestration |   |— gate-530/               # Gate 530
compliance evaluation sidecar |   |— evidence-fabric/        # Evidence
Package signing and commitment service |   |— rag-vault/      #
RAG Vault retrieval service |   |— cos-plus/                 # COS+ Database
driver and schema migrations |   |— shared/                  # Shared
types, utilities, canonical schemas |— infra/ |   |— helm/
# Helm chart for full-stack deployment |   |— kubernetes/    #
Base Kubernetes manifests |   |— terraform/                  # Infrastructure-
as-code for cloud provisioning |— docs/ |   |— volumes/      #
Vol I, II, III (public-facing architecture documents) |   |— engineering-
body/           # EB Docs 1-6 (NDA-classified internal specifications) |—
scripts/       |— db/                                         # Migration runners, seed scripts
|— ops/         # Operational runbooks as executable scripts
```

1.2 Package Dependency Graph

Package dependencies are strictly acyclic. The dependency structure is as follows:

```
middleware-engine |— gate-530 (via Unix domain socket IPC) |—
evidence-fabric (via gRPC) |— rag-vault (via gRPC) |— cos-
plus (via COS+ driver — ios_app role) |— shared gate-530 |—
cos-plus (via COS+ driver — session cache reads) |— shared
evidence-fabric |— cos-plus (via COS+ driver — audit_writer role,
WORM commits) |— shared rag-vault |— cos-plus (via COS+
```

driver – HNSW vector index) └─ shared cos-plus └─ shared
(leaf service – PostgreSQL wire protocol only) shared └─ (no internal dependencies)

Architectural Invariant

No circular dependencies are permitted. The middleware-engine is the sole orchestration package — it is the only package authorized to invoke multiple peer services. All other packages (gate-530, evidence-fabric, rag-vault, cos-plus) are consumed services and must not call back into middleware-engine or into each other. The shared package has zero internal dependencies and is the foundation of all inter-package type contracts.

Package	Role	Depends On	Consumed By
middleware-engine	Orchestrator	gate-530, evidence-fabric, rag-vault, cos-plus, shared	External callers (REST/gRPC)
gate-530	Compliance evaluation sidecar	cos-plus, shared	middleware-engine
evidence-fabric	Signing & commitment service	cos-plus, shared	middleware-engine
rag-vault	Retrieval service	cos-plus, shared	middleware-engine
cos-plus	Database driver (leaf)	shared, PostgreSQL	middleware-engine, gate-530, evidence-fabric, rag-vault
shared	Type contracts & utilities (foundation)	None	All packages

Section 2 — Middleware Engine — Module Specification

2.1 Entry Point & Transport Layer

The Middleware Engine exposes two parallel transport adapters. Both normalize inbound requests to a canonical `InferenceRequest` object before passing control to the Layer Orchestrator. TLS termination is handled at the ingress layer (not within the application process); mTLS is enforced between all internal service-to-service calls.

- **REST Adapter:** Express.js HTTP/1.1 server. All inbound requests are validated against the OpenAPI 3.1 schema before deserialization. Invalid schema yields HTTP 400 before the orchestrator is invoked.
- **gRPC Adapter:** gRPC server with protobuf schema enforcement. Bidirectional streaming is supported for batch inference workloads. Unary calls are used for standard single-query inference.

InferenceRequest — Canonical Schema

```
interface InferenceRequest {  request_id:      string;    // UUID v4, client-supplied or server-generated  tenant_id:      string;    // UUID v4, validated against tenant registry  query_payload:  string;    // Raw query text or JSON-encoded structured query  query_format:  'text' | 'json' | 'structured';  classification_assertion:  ClassificationLevel;  // Caller's asserted clearance level  session_id?:  string;    // Optional: for multi-turn context  metadata:  RequestMetadata; } interface RequestMetadata {  source_system:  string;    // Identifier of originating system  source_ip:      string;    // For data residency enforcement at Gate 530  user_agent:      string;    api_version:  string;  idempotency_key?:  string;  // Optional: enables safe client-side retry }
```

2.2 Layer Orchestrator

The Layer Orchestrator is the core module of `middleware-engine`. It implements a sequential pipeline executor that carries state across all seven processing layers via an `ExecutionContext` object instantiated per request. Key behaviors:

- Layers L1 through L7 are invoked in strict sequence. A **BLOCK** or unrecoverable error from any layer short-circuits the pipeline; no subsequent layers are invoked.
- The orchestrator records an entry timestamp and exit timestamp for every layer in `layer_timestamps`, enabling per-layer latency attribution in distributed traces.
- A **QUARANTINE** decision from Gate 530 (L5) parks the `ExecutionContext` in an in-memory quarantine store (keyed by `quarantine_id`, configurable TTL defaulting to 24 hours). Execution resumes from L6 when a compliance reviewer clears the quarantine via the Compliance Review API.

ExecutionContext — Structure

```
interface ExecutionContext {  trace_id:          string;          //
Assigned at L4; globally unique across all tenants  request:
InferenceRequest;  l1_output?:          CanonicalQueryObject;  l2_output?:
SemanticRepresentation;  l3_output?:          OntologicalQueryGraph;
evidence_package: EvidencePackage;      // Built progressively L4 → L7
gate_decision?:   GateDecisionRecord;  rag_context?:          ContextManifest;
synthesis_output?: SynthesisResult;  layer_timestamps: Record<string,
Date>; // Keys: 'L1_enter', 'L1_exit', ..., 'L7_exit'  pipeline_state:
'running' | 'blocked' | 'quarantined' | 'complete'; }
```

2.3 Layer Module Interfaces (L1-L7)

Each layer is an independently bounded module with a defined input contract, output contract, timeout budget, and timeout behavior. Layer timeout violations are deterministic — no retries are performed within a single inference pipeline execution.

Performance Note

Total end-to-end P99 budget is **555ms** (sum of all layer timeout budgets).

Production P99 is measured at **380ms** due to overlap in L1-L3 preprocessing and L6 pre-fetch parallelism. All timeout budgets are independently configurable via the global configuration schema (Section 9.1).

Layer	Module	Input	Output	Timeout Budget	On Timeout / Error
L1	IngestionModule	InferenceRequest	CanonicalQueryObject	10 ms	Reject — HTTP 400
L2	SemanticParserModule	CanonicalQueryObject	SemanticRepresentation	50 ms	Reject — HTTP 422
L3	OntologicalMapperModule	SemanticRepresentation	OntologicalQueryGraph	80 ms	Reject — HTTP 422
L4	EvidenceAnchorModule	OntologicalQueryGraph	EvidencePackage (skeleton)	5 ms	Reject — HTTP 500
L5	Gate530Module	OntologicalQueryGraph + EvidencePackage	GateDecisionRecord	60 ms	Synthetic BLOCK (gate_timeout: true)
L6	ContextAssemblerModule	OntologicalQueryGraph + GateDecisionRecord	ContextManifest	150 ms	Reject — HTTP 503
L7	SynthesisModule	ContextManifest + EvidencePackage	SynthesisResult	200 ms	Reject — HTTP 503

2.4 Error Handling & Retry Policy

Error handling is designed to guarantee auditability under all failure conditions. The following rules are invariant:

- **L1-L4 errors** are classified as client errors (4xx) or infrastructure errors (5xx). No Evidence Package is committed for errors occurring before L4 completes, because no `trace_id` has been assigned.
- **Gate 530 timeout (L5)** produces a synthetic BLOCK decision with `gate_timeout = true` in the `GateDecisionRecord`. Conservative policy: a timeout is treated as a block. No inference proceeds past an unresolved gate evaluation.
- **L6-L7 errors** produce HTTP 503. Crucially, the Evidence Package skeleton (committed at L4) has already been written to COS+ before the error is returned. Every attempted inference — including failed ones — is permanently recorded in the audit log. There are no silent failures.
- **Retry policy:** No automatic retries are performed within the pipeline. Retry logic is the caller's responsibility. The `idempotency_key` field in `RequestMetadata` enables callers to safely retry without risk of double-commitment: the orchestrator deduplicates on `idempotency_key` within a 24-hour window.

Section 3 — Gate 530 — Internal Module Specification

Cross-Reference

Full coverage of all 127 compliance dimensions — including detection functions, scoring algorithms, dimension thresholds, and inter-dimension dependency graphs — is provided in **EB Doc 4: Gate 530 Compliance Dimension Matrix**. This section covers Gate 530's software architecture and integration interfaces only.

3.1 IPC Interface

Gate 530 communicates with the Middleware Engine exclusively via **Unix domain socket (IPC)** — not TCP. This design choice eliminates network stack overhead, kernel TCP processing, and loopback routing, achieving sub-1ms transport latency on the same node. Gate 530 runs as a sidecar container in the same Kubernetes pod as the Middleware Engine, sharing the same node filesystem namespace for the Unix socket.

Gate530EvalRequest Schema

```
interface Gate530EvalRequest {  trace_id:      string;
// From ExecutionContext – globally unique  tenant_id:      string;
// UUID v4  ontological_graph:      OntologicalQueryGraph;
evidence_skeleton:      EvidencePackage;      // Skeleton from L4
regulatory_profile_id:  string;      // Active profile for this
tenant  session_context_id?:  string;      // For Group 9
behavioral_dimensions } interface Gate530EvalResponse {  gate_decision_id:
string;      // UUID v4 – permanent identifier  decision_state:
'PASS' | 'BLOCK' | 'QUARANTINE';  dimension_scores:      DimensionScore[];
// All 127 dimension evaluations  violated_dimensions:      number[];
// Dimension IDs with score below threshold  evaluation_duration_ms:
number;  gate_decision_record:      GateDecisionRecord;  // Pre-signed by
Gate 530 sidecar }
```

3.2 Dimension Evaluation Engine

The Dimension Evaluation Engine is the computational core of Gate 530. Its architecture is designed for correctness, determinism, and maximum evaluation throughput within the 60ms timeout budget.

- **Dependency graph:** All 127 dimensions are organized in a directed acyclic dependency graph. Dimensions that consume outputs of other dimensions (e.g., a behavioral anomaly dimension that depends on a prior intent classification dimension) declare their dependencies in the dimension registry.
- **Topological sort:** At service startup, the engine computes a topological sort of the dependency graph. This ordering is cached and applied to every evaluation request.

- **Parallel evaluation:** Independent dimensions (those with no shared dependencies in the current evaluation batch) are evaluated concurrently within a thread pool. Dependent dimensions are evaluated strictly after their dependencies complete.
- **Pure functions:** Each dimension evaluation is a pure function with the signature: `(OntologicalQueryGraph, SessionContext, TenantConfig) → DimensionScore`. No side effects are permitted during evaluation. All writes (gate decision record, behavioral feature update) occur after the full evaluation is complete.
- **Threshold evaluation:** After all dimension scores are computed, the engine applies the tenant's active regulatory profile to determine per-dimension pass/fail. Aggregate decision logic (PASS / BLOCK / QUARANTINE) is defined in the regulatory profile and applied to the dimension-level results.

3.3 Session Context Cache

Group 9 behavioral dimensions (dimensions 103–127) require session-level context: recent query history, per-user access patterns, query velocity metrics, and temporal anomaly indicators. This context is maintained in a shared Redis cluster.

Property	Value
Cache backend	Redis cluster (Sentinel-managed, 3-node)
Scope	Shared across all Orchestration tier nodes — not per-node
Cache key	<code>tenant_id + ":" + user_identity_hash</code>
TTL	4 hours (sliding window, refreshed on each access)
Read latency (P99)	< 2ms
Write timing	After gate decision is finalized — never during evaluation
Eviction policy	allkeys-lru with TTL enforcement

3.4 Quarantine Queue Integration

When Gate 530 returns a `QUARANTINE` decision, the following sequence is executed atomically:

1. Gate 530 writes the quarantine record to COS+ (`quarantine` table, `INSERT` via the `audit_writer` role), including all 127 dimension scores, the violated dimension list, the `trace_id`, and the regulatory profile snapshot.
2. Gate 530 returns the `Gate530EvalResponse` with `decision_state: 'QUARANTINE'` and a populated `gate_decision_id` to the Middleware Engine.
3. The Middleware Engine parks the full `ExecutionContext` in the in-memory quarantine store, keyed by `quarantine_id` (equivalent to `gate_decision_id`). Default TTL: 24 hours. Configurable via `quarantine_ttl_hours` in tenant configuration.
4. The Middleware Engine returns an HTTP 202 Accepted response to the caller, with body containing the `quarantine_id` and a `review_required` flag.
5. When a compliance reviewer submits a decision via `POST /v1/compliance/queue/{quarantine_id}/clear` or `/block`, the Middleware Engine retrieves the parked `ExecutionContext` from the quarantine store and either resumes execution from L6 (on `CLEAR`) or returns a definitive `BLOCK` response (on `BLOCK`).

Section 4 — Evidence Fabric — Internal Module Specification

Cross-Reference

Full cryptographic specification — including Ed25519 key lifecycle, JCS canonicalization algorithm, Evidence Package wire format, and WORM commitment protocol — is provided in **EB Doc 2: Cryptographic Audit & Evidence Fabric Specification**. This section covers the Evidence Fabric's software architecture and integration interfaces only.

4.1 Signing Service Architecture

The Evidence Fabric runs as a dedicated microservice (`packages/evidence-fabric`). It is called by the Middleware Engine at exactly two points in the pipeline via gRPC:

6. **L4 — CreateEvidenceSkeleton:** Called immediately after the `trace_id` is assigned. Parameters: `trace_id`, `query_hash` (SHA-256 of the canonical query payload), `source_candidates` (initial RAG candidate set), `model_fingerprint` (version hash of the active inference model). Returns: an `EvidencePackage` skeleton with all L4 fields populated and signed at the skeleton level.
7. **L7 — FinalizeAndCommit:** Called after synthesis is complete. Parameters: the complete `EvidencePackage` (with all L4-L7 fields populated), `output_hash` (SHA-256 of the synthesis output), `confidence_score`. Actions performed: (1) JCS canonical serialization of the complete package, (2) Ed25519 signing, (3) WORM commit to COS+ via the `audit_writer` role. Returns: `evidence_package_id` (the permanent, immutable identifier for the committed package).

Auditability Guarantee

The Evidence Package skeleton is committed to COS+ at L4, before Gate 530 evaluation, before retrieval, and before synthesis. This means every inference attempt — regardless of gate outcome — generates a permanent, tamper-evident audit record. The L7 finalization appends the synthesis outcome to the pre-existing skeleton record; it does not replace it. Both the skeleton commit and the finalization

commit are WORM (Write Once, Read Many) operations.

4.2 gRPC Service Definition

```
syntax = "proto3"; package ios_plus.evidence_fabric.v1; service EvidenceFabric { // Create a skeleton Evidence Package at L4 (pre-gate, pre-synthesis) rpc CreateEvidenceSkeleton (EvidenceSkeletonRequest) returns (EvidencePackage); // Append the Gate 530 decision record to the in-flight Evidence Package rpc AppendGateDecision (AppendGateRequest) returns (EvidencePackage); // Finalize, sign (Ed25519), and WORM-commit the complete package at L7 rpc FinalizeAndCommit (FinalizeRequest) returns (SignedEvidencePackageRef); // Verify the cryptographic integrity of a committed package rpc VerifyPackage (VerifyRequest) returns (VerifyResponse); // Retrieve a committed, signed Evidence Package by ID rpc GetPackage (GetPackageRequest) returns (SignedEvidencePackage); }
```

4.3 Commitment Latency Budget

The `FinalizeAndCommit` RPC must complete within **25ms at P99**, as it is included within the L7 timeout budget of 200ms. The latency breakdown and margin are as follows:

Operation	Budget (P99)
JCS canonical serialization of complete EvidencePackage	~2 ms
Ed25519 signing operation	~1 ms
COS+ WORM commit (INSERT via audit_writer role)	~8 ms
gRPC network round-trip to evidence-fabric sidecar	~5 ms
Subtotal	~16 ms
Margin	~9 ms
Total P99 Budget	25 ms

Section 5 — RAG Vault — Internal Module Specification

Cross-Reference

Full retrieval pipeline specification — including HNSW configuration, BM25 lexical layer, RRF fusion algorithm, context packing, and ingestion pipeline — is provided in **EB Doc 5: RAG Vault Retrieval Pipeline Specification**. This section covers integration interfaces only.

5.1 gRPC Interface

```
syntax = "proto3"; package ios_plus.rag_vault.v1; service RagVault { //
Retrieve context chunks for an ontological query graph (called at L6)  rpc
Retrieve (RetrievalRequest)      returns (ContextManifest); // Ingest
documents into the RAG Vault (streaming – supports bulk ingestion)  rpc
Ingest (stream IngestRequest)   returns (IngestResponse); // Refresh
TTL on a document partition or specific chunk  rpc RefreshTTL
(RefreshRequest)                returns (RefreshResponse); // Retrieve a specific
chunk by ID (for Evidence Package construction)  rpc GetChunk (ChunkRequest)
returns (Chunk); }
```

5.2 RetrievalRequest Schema

```
interface RetrievalRequest {  trace_id:          string; //
Propagated from ExecutionContext  ontological_graph: OntologicalQueryGraph;
// L3 output – drives vector query construction  gate_decision_id:  string;
// Must reference a PASS-state gate decision
// RAG Vault validates this independently  clearance_level:
ClassificationLevel; // Sourced from the gate decision record
token_budget:          number; // Maximum tokens for the assembled
context window  tenant_id:          string; // Scopes
retrieval to tenant's vault partition }
```

Security Invariant

The RAG Vault independently validates that the supplied `gate_decision_id`

references a decision in PASS state before performing any retrieval. A BLOCK or QUARANTINE gate decision ID will cause the RAG Vault to reject the request with an authorization error, regardless of the caller's asserted clearance level. This is a defense-in-depth control — the Middleware Engine Layer Orchestrator also prevents L6 invocation on non-PASS gate decisions.

5.3 ContextManifest Schema

```
interface ContextManifest {  chunks:                ContextChunk[];
total_tokens:            number;                // Sum of tokens across all included
chunks  retrieval_duration_ms: number;  retrieval_strategy:    'dense' |
'hybrid'; // 'hybrid' = dense + BM25 with RRF fusion } interface
ContextChunk {  chunk_id:                string;                //
Permanent chunk identifier  text:                string;
// Chunk text content  source_id:                string;                //
Source document identifier  source_metadata:    SourceMetadata;
// Title, author, date, classification  gate_stamp:    string;
// Gate decision ID that cleared this chunk  confidence_band:    'HIGH' |
'MEDIUM' | 'LOW'; // Retrieval confidence tier  classification_level:
ClassificationLevel;    // Chunk's own classification level
vector_distance:        number;                // Distance from query
embedding (lower = closer) }
```

Section 6 — COS+ Database — Driver Specification

Cross-Reference

Full schema, complete DDL, role architecture, trigger code, partitioning strategy, and migration runbook are provided in **EB Doc 3: COS+ Database Schema & Role Architecture**. This section covers only the COS+ client driver configuration used by consuming services.

6.1 Connection Pool Architecture

Each service maintains its own isolated connection pool, bound to its designated database role. Roles are least-privilege: `ios_app` has no write access to audit tables; `audit_writer` has no read access to operational tables; `rag_reader` has no access to audit or tenant tables. All connection strings are injected at runtime via Kubernetes Secrets. Connection strings are never hardcoded in application code or committed to the repository.

Service	DB Role	Pool Size	Max Overflow	Timeout (acquire)
middleware-engine	ios_app	20	10	5 s
evidence-fabric	audit_writer	10	5	5 s
rag-vault	rag_reader	15	5	5 s
compliance-api	audit_reader	5	2	10 s

6.2 Query Patterns by Role

Service	Role	Primary Query Pattern	Index Used
middleware-engine	ios_app	SELECT on operational tables, filtered by <code>tenant_id</code> + <code>classification_level</code>	Primary compliance index (<code>tenant_id</code> , <code>classification_level</code>)
evidence-fabric	audit_writer	INSERT ONLY on audit log and evidence package tables (WORM — no UPDATE, no DELETE)	N/A (append-only; no read queries)
rag-vault	rag_reader	Vector proximity search via HNSW	<code>vector_embedding</code> HNSW index

Service	Role	Primary Query Pattern	Index Used
		index; chunk retrieval by ID	
compliance-api	audit_reader	SELECT on audit log by trace_id, date range, tenant_id; paginated	Composite index: timestamp + tenant_id

Section 7 — Shared Package — Type Definitions

The `shared` package is the single source of truth for all inter-service type contracts. No service may define its own version of a type that is defined in `shared`. All canonical types are exported from `packages/shared/src/types/index.ts`.

7.1 ClassificationLevel Enum

```
typedef enum ClassificationLevel {
  PUBLIC          = 0, // No access
restriction      INTERNAL      = 1, // Internal use only; authenticated
employees       CONFIDENTIAL   = 2, // NDA required; restricted distribution
RESTRICTED      = 3, // Enhanced NDA + role-based access control
TOP_RESTRICTED  = 4 // Maximum restriction; named-individual access only }
ClassificationLevel;
```

7.2 Core Shared Types

```
// — CanonicalQueryObject
// _____ // Output of L1
(IngestionModule). Normalized form of the raw query payload. interface
CanonicalQueryObject {
  query_id: string; // Internal ID
  (may differ from request_id) normalized_text: string; //
  Lowercased, whitespace-normalized query text query_format: 'text' |
  'json' | 'structured'; token_count: number; detected_language:
  string; // BCP 47 language tag hash: string;
  // SHA-256 of normalized_text tenant_id: string;
  classification_assertion: ClassificationLevel; } // —
```

SemanticRepresentation

```
// Output of L2
(SemanticParserModule). Semantic decomposition of the query. interface
SemanticRepresentation { intent_class: IntentClass; // Primary
intent taxonomy node intent_confidence: number; // 0.0-1.0
confidence score entities: SemanticEntity[]; relations:
SemanticRelation[]; topic_vector: number[]; // Dense
embedding vector (1536-dim) } type IntentClass = | 'information_retrieval'
| 'document_synthesis' | 'comparison_analysis' | 'decision_support' |
'code_generation' | 'data_extraction' | 'system_query' | 'unknown';
interface SemanticEntity { entity_id: string; surface_form: string;
// Raw text of the entity as it appears in the query entity_type: string;
// e.g., 'ORGANIZATION', 'REGULATION', 'PERSON', 'CONCEPT' confidence:
number; } interface SemanticRelation { subject_entity_id: string;
predicate: string; object_entity_id: string; confidence:
number; } // — OntologicalQueryGraph

// Output of L3
(OntologicalMapperModule). Graph representation of the query. interface
OntologicalQueryGraph { graph_id: string; nodes:
OntologyNode[]; edges: OntologyEdge[]; topic_clusters: string[];
// IDs of matched ontology clusters regulatory_tags: string[]; //
Matched regulatory domain tags (e.g., 'HIPAA', 'SOX') } interface
OntologyNode { node_id: string; ontology_uri:
string; // Canonical URI in the IOS+ ontology namespace label:
string; node_type: 'concept' | 'entity' | 'regulation' | 'process' |
'artifact'; attributes: Record<string, string>; } interface
OntologyEdge { edge_id: string; source_node: string;
target_node: string; relation_type: string; // e.g.,
'governs', 'requires', 'produces', 'modifies' weight: number; } //
— GateDecisionRecord

// Pre-signed record
returned by Gate 530. Permanently stored in COS+. interface
GateDecisionRecord { gate_decision_id: string; // UUID v4
— permanent identifier trace_id: string; tenant_id:
string; regulatory_profile_id: string; decision_state: 'PASS' |
'BLOCK' | 'QUARANTINE'; dimension_scores: DimensionScore[]; // All
127 dimension evaluations violated_dimensions: number[]; //
Dimension IDs below threshold gate_timeout: boolean; //
true if decision is synthetic (timeout) evaluation_duration_ms: number;
evaluated_at: string; // ISO 8601 UTC timestamp
signature: string; // Ed25519 signature (base64url)
over JCS form signing_key_id: string; // Key identifier
for rotation tracking } interface DimensionScore { dimension_id:
number; // 1-127 dimension_group: number; // 1-9 (group
membership) score: number; // 0.0-1.0 normalized score
threshold: number; // Effective threshold for this
tenant/profile passed: boolean; evaluation_ms: number; } //
— EvidencePackage

// Built
progressively L4-L7. Finalized and WORM-committed by Evidence Fabric.
interface EvidencePackage { evidence_package_id?: string; //
Assigned by Evidence Fabric on FinalizeAndCommit trace_id:
string; // Assigned at L4 tenant_id: string; // L4
fields (skeleton) query_hash: string; // SHA-256 of
canonical query payload model_fingerprint: string; // Version
hash of the active inference model source_candidates: string[]; //
Initial RAG candidate IDs skeleton_timestamp: string; // ISO 8601
UTC — time of skeleton creation skeleton_signature?: string; //
Ed25519 signature over skeleton (pre-gate) // L5 fields (appended after
Gate 530) gate_decision_id?: string; gate_decision_state?: 'PASS' |
```

```

'BLOCK' | 'QUARANTINE'; // L6 fields (appended after context assembly)
context_chunk_ids?: string[]; // IDs of all chunks included in context
context_token_count?: number; retrieval_strategy?: 'dense' | 'hybrid';
// L7 fields (finalized at synthesis) output_hash?:
string; // SHA-256 of synthesis output confidence_score?: number;
// 0.0-1.0 synthesis confidence finalized_at?: string; // ISO
8601 UTC – time of finalization final_signature?: string; //
Ed25519 signature over complete JCS-canonical form signing_key_id?:
string; committed_to_cos?: boolean; // true after WORM commit
confirms } // — SynthesisResult

// Output of L7
(SynthesisModule). interface SynthesisResult { synthesis_id:
string; output_text: string; // Generated synthesis output
output_format: 'text' | 'json' | 'structured'; confidence_score:
number; // 0.0-1.0 aggregate confidence model_id:
string; // Identifier of the inference model used
model_fingerprint: string; // Version hash (must match evidence
skeleton) output_tokens: number; source_chunk_ids: string[];
// Chunks that contributed to this output synthesis_duration_ms: number; }
// — InferenceResponse

// The final object
returned to external callers on the Inference API. interface
InferenceResponse { request_id: string; // Echoed from
InferenceRequest trace_id: string; // Globally unique;
use for audit lookups evidence_package_id: string; // Permanent
ID; use for evidence retrieval gate_decision_id: string; //
Permanent ID; use for gate decision retrieval status:
'complete' | 'blocked' | 'quarantined' | 'error'; output?:
SynthesisResult; // Present only on status: 'complete' block_reason?:
string; // Human-readable reason; present on 'blocked'
quarantine_id?: string; // Present on status: 'quarantined'
pipeline_duration_ms: number; // End-to-end wall time api_version:
string; }

```

Section 8 — Internal API Surface

8.1 Inference API (External-Facing)

The Inference API is the primary interface for authorized callers. All endpoints require a valid API key bound to a provisioned tenant. Rate limits and quotas are enforced per tenant at the ingress layer.

Method	Endpoint	Description	Auth
POST	/v1/infer	Submit an	API Key + Tenant

Method	Endpoint	Description	Auth
		inference request. Returns InferenceResponse on complete/blocked, or HTTP 202 with quarantine_id on quarantine.	
GET	/v1/evidence/{evidence_package_id}	Retrieve a committed, signed Evidence Package by its permanent ID. Returns SignedEvidencePackage.	API Key + Tenant
GET	/v1/gate/{gate_decision_id}	Retrieve a gate decision record by its permanent ID. Returns GateDecisionRecord including all dimension scores.	API Key + Tenant
POST	/v1/quarantine/{quarantine_id}/review	Submit a reviewer decision (CLEAR or BLOCK) for a quarantined query. Body: { decision: 'CLEAR' 'BLOCK', reviewer_id: string, notes?: string }.	Reviewer Role + MFA

8.2 Compliance Review API (Internal / Admin)

The Compliance Review API is restricted to compliance officers and system administrators. It is served on a separate internal port (not exposed at the public ingress) and requires role-based authentication with MFA.

Method	Endpoint	Description
GET	/v1/compliance/queue	List all quarantine queue

Method	Endpoint	Description
		entries for the authenticated tenant. Supports pagination (?page=, ?limit=) and filter by date range and decision state.
GET	/v1/compliance/queue/{quarantine_id}	Inspect a specific quarantined query: full ontological graph, all 127 dimension scores, regulatory profile snapshot at time of evaluation.
POST	/v1/compliance/queue/{quarantine_id}/clear	PASS reviewer decision. Resumes inference pipeline from L6 using parked ExecutionContext. Body: { reviewer_id, justification }. Reviewer identity is appended to the Evidence Package before finalization.
POST	/v1/compliance/queue/{quarantine_id}/block	BLOCK reviewer decision. Terminates the quarantined query permanently. Body: { reviewer_id, justification }. Updates the gate decision record with BLOCK state and commits final Evidence Package.
GET	/v1/compliance/audit-log	Paginated audit log query. Filters: ?tenant_id=, ?trace_id=, ?from_date=, ?to_date=, ?decision_state=, ?dimension_id=.

8.3 Admin API (Infrastructure Only)

The Admin API is restricted to infrastructure operators and is served on the internal management port only. It is not reachable from the public network or from tenant contexts.

Method	Endpoint	Description
GET	/v1/admin/health	Health check with per-component status: middleware-engine, gate-530, evidence-fabric, rag-vault, cos-plus, Redis. Returns HTTP 200 (healthy) or 503 (degraded) with component-level detail.
GET	/v1/admin/metrics	Prometheus-format metrics endpoint. Exposes: request rate, per-layer latency histograms, gate decision distribution (PASS/BLOCK/QUARANTINE), evidence commitment latency, quarantine queue depth.
POST	/v1/admin/key-rotate	Trigger a signing key rotation event. Generates a new Ed25519 key pair, registers the new key in the key registry, and gracefully transitions signing operations. Old key remains valid for verification. Requires operator MFA.
POST	/v1/admin/profile-reload	Reload a tenant's regulatory profile from the versioned profile store. Body: { tenant_id, profile_version }. Hot-reload — does not require service restart. In-flight evaluations complete against the prior profile version.

Section 9 — Configuration Schema

9.1 Global Configuration (config/global.yaml)

```
#####
# # IOS+ Global Configuration Schema # File: config/global.yaml # All values
# shown are defaults. Override via Helm values or environment variables.
#####
# service:  name: ios-plus-middleware  environment: production
# production | staging | development  log_level: info
# debug | info | warn | error  trace_sampling_rate: 0.05  #
Fraction of requests to sample for distributed tracing  transport:  rest:
port: 8080  request_timeout_ms: 600  # Maximum time to hold an
open HTTP connection  max_body_size_bytes: 1048576  # 1 MB  grpc:
port: 9090  max_message_size_bytes: 4194304  # 4 MB  tls:
termination: ingress  # TLS is terminated at ingress; not in-
process  mtls_between_services: true  # Mutual TLS enforced for all
service-to-service calls  layer_timeouts_ms:  l1_ingestion: 10
l2_semantic_parser: 50  l3_ontological_mapper: 80  l4_evidence_anchor: 5
l5_gate_530: 60  l6_context_assembler: 150  l7_synthesis: 200
feature_flags:  quarantine_enabled: true  # If false, QUARANTINE
decisions behave as BLOCK  behavioral_monitoring_enabled: true  # If false,
Group 9 dimensions (103-127) are skipped  evidence_skeleton_on_error: true
# If true, skeleton is committed even on L1-L3 errors
idempotency_dedup_window_hours: 24  # Window for idempotency_key
deduplication  quarantine:  store_type: in_memory  #
in_memory | redis (use redis for multi-replica)  default_ttl_hours: 24
max_queue_depth_per_tenant: 500  redis:  session_cache:  host: redis-
cluster.internal  port: 6379  db: 0  tls: true
connect_timeout_ms: 500  command_timeout_ms: 2000  pool_size: 10
evidence_fabric:  grpc_endpoint: evidence-fabric.internal:9090
commitment_timeout_ms: 25  verify_on_retrieve: true  rag_vault:
grpc_endpoint: rag-vault.internal:9090  default_token_budget: 4096
retrieval_timeout_ms: 120  gate_530:  ipc_socket_path:
/tmp/ios-plus/gate530.sock  eval_timeout_ms: 60  timeout_policy: block
# block | pass (always block on timeout)  cos_plus:  host: cosplus-
primary.internal  port: 5432  database: cosplus_prod  ssl_mode: verify-
full  ssl_cert_path: /etc/certs/cos-plus-client.crt  observability:
metrics_port: 9091  health_port: 9092  tracing_endpoint: http://otel-
collector.internal:4318  log_format: json  # json | text
```

9.2 Tenant Configuration Schema

```
#####
# # IOS+ Tenant Configuration Schema # Per-tenant. Stored versioned in COS+
# tenant_config table. # Retrieved by middleware-engine at request time
# (cached, invalidated on reload).
#####
# tenant_id: "550e8400-e29b-41d4-a716-446655440000"  # UUID v4 – immutable
tenant_name: "Acme Corp – Regulated Division"  tenant_tier: enterprise
```

```
# standard | enterprise | sovereign active_regulations: - HIPAA - SOX
- GDPR - ISO27001 jurisdiction_flags: - US_DOMESTIC - EU_GDPR -
data_residency_enforced: true data_residency_regions: - us-east-1 - eu-
west-1 classification_max_level: RESTRICTED # Maximum
classification this tenant may query dimension_thresholds: # Per-dimension
threshold overrides (keyed by dimension ID). # Omitted dimensions use
regulatory profile default thresholds. # Example: tighten threshold on
dimension 42 (cross-tenant access detection). 42: 0.85 103: 0.90 115:
0.75 quarantine_queue_id: "queue-acme-regulated-01" quarantine_ttl_hours:
48 # Override global default of 24 hours
rag_vault_partition_ids: - "partition-acme-internal-docs" - "partition-
acme-regulatory-corpus" - "partition-shared-regulatory-baseline"
retention_profiles: evidence_packages_days: 2555 # 7
years (SOX compliance) gate_decisions_days: 2555 audit_log_days: 3650
# 10 years api_rate_limits: requests_per_minute: 300 burst_capacity: 60
```

Section 10 — Document Lineage

The following table maps each IOS+ Engineering Body document to the specific section or sections of this document (EB Doc 1) that describe the corresponding implementation interfaces. Engineers seeking deeper specification for any layer should consult the indicated document.

From (This Document)	Relates To	Relationship / Depth Added
EB Doc 1 (this document) — All Sections	Vol I — Architecture Overview	Implementation depth: EB Doc 1 specifies <i>how</i> the architecture described in Vol I is built. Vol I describes intent; EB Doc 1 specifies the contracts.
Section 4 — Evidence Fabric	EB Doc 2 — Cryptographic Audit & Evidence Fabric Specification	EB Doc 2 provides full Ed25519 key lifecycle, JCS canonicalization algorithm, Evidence Package wire format, and WORM protocol — the cryptographic detail behind the interfaces described in Section 4.
Section 6 — COS+ Database Driver	EB Doc 3 — COS+ Database Schema & Role	EB Doc 3 provides full DDL, role permission

From (This Document)	Relates To	Relationship / Depth Added
	Architecture	grants, trigger code, partitioning, and migration runbook — the database internals underlying the driver patterns described in Section 6.
Section 3 — Gate 530	EB Doc 4 — Gate 530 Compliance Dimension Matrix	EB Doc 4 defines all 127 dimensions with detection functions, scoring algorithms, threshold defaults, and inter-dimension dependencies — the evaluation logic inside the engine described in Section 3.
Section 5 — RAG Vault	EB Doc 5 — RAG Vault Retrieval Pipeline Specification	EB Doc 5 specifies HNSW configuration, BM25 lexical layer, RRF fusion algorithm, context packing, and ingestion pipeline — the retrieval internals behind the interfaces described in Section 5.
Section 9 — Configuration Schema	EB Doc 6 — Deployment, Operations & Recovery Runbook	EB Doc 6 covers Kubernetes deployment, Helm chart values, monitoring/alerting configuration, incident response, and disaster recovery — the operational context for the configuration schemas defined in Section 9.

Appendix A — Full InferenceResponse Schema

The `InferenceResponse` is the terminal object returned to all external callers upon completion of a successful inference pipeline. It is also returned (with appropriate status) on BLOCK and QUARANTINE outcomes. The schema is authoritative as defined in `packages/shared/src/types/index.ts`.

```
// — InferenceResponse — Full Schema
// Returned by POST /v1/infer //
Defined in: packages/shared/src/types/index.ts interface InferenceResponse {
// — Request Echo —
request_id: string; // Echoed from
InferenceRequest.request_id tenant_id: string; // Echoed
from InferenceRequest.tenant_id // — Audit References (always present)
trace_id:
string; // Globally unique; use for all audit lookups
evidence_package_id: string; // Permanent ID for evidence retrieval
gate_decision_id: string; // Permanent ID for gate decision
retrieval // — Pipeline Outcome

status: 'complete' |
'blocked' | 'quarantined' | 'error'; // — Synthesis Output (present only
on status: 'complete')
output?: { text:
string; // Generated synthesis output text format:
'text' | 'json' | 'structured'; confidence_score: number; //
0.0-1.0 model_id: string; model_fingerprint: string;
output_tokens: number; source_chunk_ids: string[]; // Chunks
contributing to this output }; // — Block Detail (present on status:
'blocked')
block_reason?: string;
// Human-readable block reason gate_timeout?: boolean; //
true if block was caused by Gate 530 timeout violated_dimensions?:
number[]; // Dimension IDs that caused the block // — Quarantine
Detail (present on status: 'quarantined')
quarantine_id?:
string; // Use with Compliance Review API review_required?:
boolean; // Always true when quarantine_id is present
quarantine_expires_at?: string; // ISO 8601 UTC — when parked context
expires // — Error Detail (present on status: 'error')
error_code?: string; //
Machine-readable error code error_message?: string; // Human-
readable error description failed_layer?: string; // e.g.,
'L6' — which layer produced the error // — Performance Telemetry
pipeline_duration_ms:
number; // End-to-end wall time (L1 enter → response sent)
layer_durations_ms?: { l1?: number; l2?: number; l3?: number;
l4?: number; l5?: number; l6?: number; l7?: number; }; // —
API Metadata
api_version: string; // e.g., '1.0' responded_at:
string; // ISO 8601 UTC timestamp }
```

Appendix B — Protobuf Service Definitions

The following proto3 service definitions cover all three gRPC services in the IOS+ platform. These definitions are the canonical contracts; generated client and server

stubs are produced from these files during the build process. Source files are located in `packages/shared/proto/`.

B.1 EvidenceFabric Service (`evidence_fabric.proto`)

```
syntax = "proto3"; package ios_plus.evidence_fabric.v1; option go_package =
"github.com/smeprotech/ios-plus/evidence_fabric/v1"; service EvidenceFabric
{   rpc CreateEvidenceSkeleton (EvidenceSkeletonRequest) returns
(EvidencePackage);   rpc AppendGateDecision (AppendGateRequest)
returns (EvidencePackage);   rpc FinalizeAndCommit (FinalizeRequest)
returns (SignedEvidencePackageRef);   rpc VerifyPackage
(VerifyRequest) returns (VerifyResponse);   rpc GetPackage
(GetPackageRequest) returns (SignedEvidencePackage); } message
EvidenceSkeletonRequest {   string trace_id = 1;   string tenant_id
= 2;   string query_hash = 3;   // SHA-256 hex of canonical query
payload   string model_fingerprint = 4;   repeated string source_candidates =
5; } message AppendGateRequest {   string trace_id = 1;   string
gate_decision_id = 2;   string decision_state = 3;   // 'PASS' | 'BLOCK'
| 'QUARANTINE'   bytes gate_decision_cbor = 4;   // CBOR-encoded
GateDecisionRecord } message FinalizeRequest {   string trace_id =
1;   bytes evidence_package = 2;   // JSON-encoded complete EvidencePackage
string output_hash = 3;   // SHA-256 hex of synthesis output   double
confidence_score = 4; } message SignedEvidencePackageRef {   string
evidence_package_id = 1;   // Permanent COS+ record ID   string committed_at
= 2;   // ISO 8601 UTC   string signature = 3;   // Ed25519 signature
(base64url)   string signing_key_id = 4; } message VerifyRequest
{   string evidence_package_id = 1; } message GetPackageRequest {   string
evidence_package_id = 1; } message VerifyResponse {   bool valid
= 1;   string failure_reason = 2;   // Present if valid = false   string
signing_key_id = 3;   string verified_at = 4; } message
SignedEvidencePackage {   string evidence_package_id = 1;   bytes
package_json = 2;   // JCS-canonical JSON of complete EvidencePackage
string signature = 3;   string signing_key_id = 4;   string
committed_at = 5; }
```

B.2 RagVault Service (`rag_vault.proto`)

```
syntax = "proto3"; package ios_plus.rag_vault.v1; option go_package =
"github.com/smeprotech/ios-plus/rag_vault/v1"; service RagVault {   rpc
Retrieve (RetrievalRequest) returns (ContextManifest);   rpc Ingest
(stream IngestRequest) returns (IngestResponse);   rpc RefreshTTL
(RefreshRequest) returns (RefreshResponse);   rpc GetChunk
(ChunkRequest) returns (Chunk); } message RetrievalRequest
{   string trace_id = 1;   bytes ontological_graph = 2;   // JSON-
encoded OntologicalQueryGraph   string gate_decision_id = 3;   // Must be
PASS state – validated by RAG Vault   int32 clearance_level = 4;   //
ClassificationLevel enum value   int32 token_budget = 5;   string
tenant_id = 6; } message ContextManifest {   repeated ContextChunk
chunks = 1;   int32 total_tokens = 2;   int32
retrieval_duration_ms = 3;   string retrieval_strategy = 4;   //
'dense' | 'hybrid' } message ContextChunk {   string chunk_id =
1;   string text = 2;   string source_id = 3;   bytes
source_metadata_json = 4;   string gate_stamp = 5;   string
confidence_band = 6;   // 'HIGH' | 'MEDIUM' | 'LOW'   int32
classification_level = 7;   double vector_distance = 8; } message
IngestRequest {   string source_id = 1;   string text =
2;   bytes source_metadata_json = 3;   int32 classification_level = 4;
string tenant_id = 5;   int32 ttl_days = 6;   // 0 }
```

```
= no TTL } message IngestResponse {   int32 chunks_created           = 1;
int32 chunks_updated           = 2;   string ingestion_id           = 3; }
message RefreshRequest { string source_id = 1; int32 ttl_days = 2; } message
RefreshResponse { bool success = 1; } message ChunkRequest { string
chunk_id = 1; } message Chunk {   string chunk_id           = 1;   string
text           = 2;   string source_id           = 3;   int32
classification_level = 4; }
```

B.3 Gate530 Service (gate_530.proto)

```
syntax = "proto3"; package ios_plus.gate530.v1; option go_package =
"github.com/smeprotech/ios-plus/gate530/v1"; // NOTE: Gate 530 communicates
via Unix domain socket IPC using this proto schema. // The gRPC transport is
configured to use the Unix socket path from global config. service Gate530 {
rpc Evaluate          (Gate530EvalRequest) returns (Gate530EvalResponse);
rpc GetDecision       (GetDecisionRequest) returns (Gate530EvalResponse);
rpc PurgeSessionCache (PurgeCacheRequest)  returns (PurgeCacheResponse);
rpc ReloadProfile     (ReloadProfileRequest) returns (ReloadProfileResponse);
} message Gate530EvalRequest {   string trace_id           = 1;   string
tenant_id           = 2;   bytes  ontological_graph_json = 3;   // JSON-
encoded OntologicalQueryGraph bytes  evidence_skeleton_json = 4;   // JSON-
encoded EvidencePackage skeleton string regulatory_profile_id = 5;
string session_context_id = 6;   // Optional; for Group 9 dimensions }
message Gate530EvalResponse {   string gate_decision_id     = 1;   string
decision_state      = 2;   // 'PASS' | 'BLOCK' | 'QUARANTINE' repeated
DimensionScore scores = 3;   repeated int32 violated_dims   = 4;   int32
evaluation_duration_ms = 5;   bytes  gate_decision_record_json = 6;   //
Pre-signed GateDecisionRecord (JSON) } message DimensionScore {   int32
dimension_id        = 1;   int32 dimension_group = 2;   double score          =
3;   double threshold = 4;   bool passed           = 5;   int32
evaluation_ms       = 6; } message GetDecisionRequest { string
gate_decision_id = 1; } message PurgeCacheRequest { string tenant_id = 1;
string user_identity_hash = 2; } message PurgeCacheResponse { bool success
= 1; } message ReloadProfileRequest { string tenant_id = 1; string
profile_version = 2; } message ReloadProfileResponse { bool success = 1;
string active_version = 2; }
```

Appendix C — OpenAPI 3.1 Summary

The following table summarizes all endpoints defined in the IOS+ OpenAPI 3.1 specification. The full OpenAPI document (docs/openapi/ios-plus-v1.yaml) is the authoritative source for request/response schemas, error codes, and security scheme definitions.

Method	Path	Request Type	Response Type	API Group
POST	/v1/infer	InferenceRequest	InferenceResponse	Inference

Method	Path	Request Type	Response Type	API Group
GET	/v1/evidence/{evidence_package_id}	—	SignedEvidencePackage	Inference
GET	/v1/gate/{gate_decision_id}	—	GateDecisionRecord	Inference
POST	/v1/quarantine/{quarantine_id}/review	ReviewDecisionRequest	InferenceResponse	Inference
GET	/v1/compliance/queue	Query params: page, limit, from_date, to_date	QuarantineQueuePage	Compliance
GET	/v1/compliance/queue/{quarantine_id}	—	QuarantineDetail	Compliance
POST	/v1/compliance/queue/{quarantine_id}/clear	ReviewDecisionRequest	InferenceResponse	Compliance
POST	/v1/compliance/queue/{quarantine_id}/block	ReviewDecisionRequest	InferenceResponse	Compliance
GET	/v1/compliance/audit-log	Query params: tenant_id, trace_id, from_date, to_date, decision_state, dimension_id, page, limit	AuditLogPage	Compliance
GET	/v1/admin/health	—	HealthStatus	Admin
GET	/v1/admin/metrics	—	Prometheus text format	Admin
POST	/v1/admin/key-rotate	KeyRotateRequest	KeyRotateResponse	Admin

Method	Path	Request Type	Response Type	API Group
POST	/v1/admin/profile-reload	ProfileReload Request	ProfileReload Response	Admin

Security Schemes

Scheme Name	Type	Applied To	Description
TenantApiKey	API Key (header: X-IOS-API-Key)	Inference API	Issued per tenant. Bound to tenant_id. Rate-limited at ingress.
ReviewerBearerMFA	Bearer JWT + MFA claim	Compliance Review API	Short-lived JWT issued after MFA verification. Scoped to reviewer role.
OperatorBearerMFA	Bearer JWT + MFA claim	Admin API	Short-lived JWT issued after MFA verification. Scoped to operator role. Internal network only.

Appendix D — Glossary

The following terms are defined as used in this document. For regulatory and compliance terminology, see also EB Doc 4 (Gate 530 Compliance Dimension Matrix). For cryptographic terminology, see EB Doc 2.

Term	Definition
CanonicalQueryObject	Output of L1 (IngestionModule). The normalized, hash-identified representation of a raw inference query. All downstream layers operate

Term	Definition
	on this object, never the raw payload.
ClassificationLevel	Five-level enumeration (PUBLIC=0 through TOP_RESTRICTED=4) that governs access to content and inference outputs. Enforced by Gate 530 and RAG Vault independently.
ContextManifest	Output of L6 (ContextAssemblerModule). The assembled set of context chunks, with token count and retrieval metadata, passed to L7 for synthesis.
DimensionScore	The result of evaluating one of Gate 530's 127 compliance dimensions against a query. Includes raw score, effective threshold, and pass/fail status.
EvidencePackage	The progressive audit record built L4→L7. Includes query hash, model fingerprint, gate decision, context chunk references, output hash, and Ed25519 signature. WORM-committed to COS+ at L4 (skeleton) and L7 (final).
ExecutionContext	The per-request state object carrying all layer inputs, outputs, and timestamps across the L1-L7 pipeline. Parked in the quarantine store on QUARANTINE decisions.
GateDecisionRecord	The pre-signed, permanently stored record of a Gate 530 evaluation. Includes all 127 dimension scores, the aggregate decision (PASS/BLOCK/QUARANTINE), and the Ed25519 signature.
IPC	Inter-Process Communication. Used for Gate 530 ↔ Middleware Engine communication via Unix domain socket to achieve sub-1ms transport latency.
JCS	JSON Canonicalization Scheme (RFC 8785). Used by the Evidence Fabric to produce a deterministic byte-for-byte serialization of the EvidencePackage before signing.
OntologicalQueryGraph	Output of L3 (OntologicalMapperModule). A directed graph of ontology nodes and edges

Term	Definition
	representing the query's meaning in terms of the IOS+ regulatory ontology. The primary input to Gate 530 evaluation.
OntologyNode	A node in the OntologicalQueryGraph. Represents a concept, entity, regulation, process, or artifact from the IOS+ ontology namespace.
OntologyEdge	A directed edge in the OntologicalQueryGraph. Represents a semantic relation (e.g., <i>governs</i> , <i>requires</i> , <i>produces</i>) between two ontology nodes.
QuarantineStore	The in-memory (or Redis-backed, for multi-replica deployments) store where ExecutionContext objects are parked pending compliance review. TTL-controlled.
RRF	Reciprocal Rank Fusion. The score fusion algorithm used by RAG Vault to combine dense vector retrieval (HNSW) and lexical retrieval (BM25) results in hybrid retrieval mode.
SemanticRepresentation	Output of L2 (SemanticParserModule). Includes intent classification, confidence score, entity list, relation list, and a 1536-dimension dense embedding vector of the query.
SignedEvidencePackageRef	The lightweight reference object returned by FinalizeAndCommit. Contains the permanent evidence_package_id, commit timestamp, Ed25519 signature, and signing key ID.
SynthesisResult	Output of L7 (SynthesisModule). The generated inference output text or structured data, with confidence score, model fingerprint, token count, and contributing chunk IDs.
InferenceRequest	The canonical input object for the IOS+ pipeline, normalized from either the REST or gRPC transport by the respective adapter. All layers consume this object via the ExecutionContext.
InferenceResponse	The terminal output object returned to

Term	Definition
	external callers. Contains the synthesis result (on COMPLETE), block reason (on BLOCK), quarantine ID (on QUARANTINE), and permanent audit reference IDs in all cases.
WORM	Write Once, Read Many. The commit policy enforced by COS+ for all Evidence Package and audit log records. UPDATE and DELETE are not permitted on WORM tables regardless of database role.
audit_writer	The COS+ database role assigned exclusively to the Evidence Fabric service. Has INSERT permission on audit and evidence tables only. No SELECT, UPDATE, or DELETE permissions on any table.
ios_app	The COS+ database role assigned to the Middleware Engine. Has SELECT on operational tables. No access to audit tables.
rag_reader	The COS+ database role assigned to the RAG Vault service. Has SELECT on vector index and chunk tables. No access to audit, evidence, or tenant configuration tables.
audit_reader	The COS+ database role assigned to the Compliance Review API. Has SELECT on audit log, evidence package, and gate decision tables. No write permissions of any kind.
Group 9 Dimensions	Gate 530 compliance dimensions 103–127. Behavioral dimensions that evaluate query patterns over time against session context. Require the Redis session context cache and are skipped if behavioral_monitoring_enabled is false.
HNSW	Hierarchical Navigable Small World. The approximate nearest-neighbor index algorithm used by RAG Vault for dense vector retrieval. Configured and documented in EB Doc 5.
Regulatory Profile	A versioned, per-tenant configuration object that maps active regulations to Gate 530 dimension thresholds and

Term	Definition
	decision aggregation logic. Hot-reloadable via the Admin API without service restart.

CONFIDENTIAL — INTERNAL ENGINEERING REFERENCE — ENHANCED NDA
REQUIRED